电商销量预测系统 · RAG 语料库与提示词

版本 V1.0 | 2026-10-13 | 语料块 40 个 | 提示词 5 组

本文档是《项目任务书 V1.0.0》§5.2 要求的「RAG 语料库：根据提供的 skill.md 生成的语料库，以及用于构建 RAG 助手的提示词」交付物。

⚠ **关于 `skill.md` 的说明**：任务书称语料库应「根据**提供的** skill.md 生成」，但该 skill.md **未随任务书提供**（本项目已在 PRD 附录 A `Q11`、开发方案附录 `Q11` 登记为待确认项）。因此本语料库**改以项目自身已冻结的知识资产为源**——即《开发技术文档》的 `BE-*` 约束体系、《项目开发实施方案》的十八条踩坑、冻结的口径与决策、以及实测数据。

这一点是刻意的：RAG 语料的价值不在"抄一份规范过来"，而在于**它是否与本项目的真实实现一致**。用项目自身文档作语料，天然不会出现"语料说 A、代码是 B"。若后续拿到 skill.md，只需按同一结构追加语料块即可。

目 录

0. 语料库设计与使用 4

0.1 语料块结构 4

0.2 主题分布（40 块） 4

0.3 机器可用产物 5

0.4 分块原则（为什么这么切） 5

1. 语料库正文 6

RC-OVW-01 项目是什么、要解决什么问题 6
RC-OVW-02 六大功能模块 6
RC-OVW-03 硬指标达成情况与实测数据 7
RC-ARC-01 系统架构与端口 7
RC-ARC-02 后端分层结构 8
RC-ARC-03 技术栈与版本（版本锁定） 9
RC-ARC-04 启动自检链与运行期硬约束 9
RC-ARC-05 接口文档离线化（Swagger / ReDoc） 10
RC-ALGO-01 算法流程四阶段 11
RC-ALGO-02 五条必须保持的算法设计铁律 11
RC-ALGO-03 序列三级降级（L1 / L2 / L3） 12
RC-ALGO-04 模型结构与超参（lstm_v1） 13
RC-ALGO-05 为什么不做在线重训练 13
RC-ALGO-06 预处理与特征工程细节 14
RC-ALGO-07 评测必须"样本外"，且基线可比 14
RC-ALGO-08 训练与调参时机（GIT-7 纪律） 15
RC-ALGO-09 已知算法局限 15
RC-MET-01 指标口径（必须按此口径回答） 16
RC-MET-02 数据集字段与校验规则 16
RC-MET-03 覆盖率披露口径 17
RC-MET-04 已知数据事实（长尾） 17
RC-DB-01 数据库对象总览 18
RC-DB-02 PRAGMA 四件套（🔒 每次连接执行） 18
RC-DB-03 迁移策略（不引入 Alembic） 19
RC-DB-04 备份与恢复（🔒 必须用 .backup） 19
RC-API-01 接口总览（30 个路径） 20
RC-API-02 统一响应包与错误码 21

RC-API-03 三条接口设计红线 22
RC-API-04 契约一致性守卫 22
RC-SEC-01 认证与会话 23
RC-SEC-02 数据隔离红线 23
RC-SEC-03 游客态与哨兵账号 24
RC-SEC-04 安全加固清单 25
RC-PERF-01 性能基线与 SLA 25
RC-PERF-02 容量估算 26
RC-PERF-03 数据清理与 TTL 三档 26
RC-PIT-01 最易踩的坑之一：副作用被异常回滚 27
RC-PIT-02 最易踩的坑之二：哨兵值 28
RC-PIT-03 最易踩的坑之三：正则扫源码不先剥注释 28
RC-PIT-04 环境类坑（换机器会重现不了，必须文档化） 29
2. 用于构建 RAG 助手的提示词 30
2.1 系统提示词（RAG 助手人设与边界） 30
2.2 检索提示词（查询改写 / 扩展） 30
2.3 回答生成提示词（带引用） 31
2.4 评测提示词（RAG 质量自检） 32
2.5 一组"必须答对"的验收问题（golden set） 33
2.6 常见的 RAG 失败模式（本项目语料下要特别注意） 34
3. 语料库的可复现性与自检 34

0. 语料库设计与使用

0.1 语料块结构

每个语料块（chunk）带四项元数据，直接对应检索增强的三类需求：

字段	作用	例
`id`	唯一标识，用于引用回指	`RC-ALGO-02`
`topic`	一级主题，用于**先过滤再检索**（比纯向量检索更稳）	算法
`source`	出处（文档 + 章节），用于**答案溯源**	《开发技术文档 V1.0》§4.5
`tags`	关键词，用于**混合检索**（BM25 / 关键词召回）	残差 切分 尖峰

0.2 主题分布（40 块）

主题	块数	编号区间
项目概览	3	`RC-OVW-*`
架构与部署	5	`RC-ARC-*`
算法与模型	9	`RC-ALGO-*`
数据与指标口径	4	`RC-MET-*`
数据库	4	`RC-DB-*`
接口与契约	4	`RC-API-*`
安全与权限	4	`RC-SEC-*`
性能与容量	3	`RC-PERF-*`
踩坑与排错	4	`RC-PIT-*`

0.3 机器可用产物

```
# 从本文档抽取语料块 → outputs/电商销量预测系统_rag_corpus.json / .jsonl
python scripts/build_rag_corpus.py


# 自检：块数、id 唯一性、元数据齐备、标签非空
python scripts/build_rag_corpus.py --check
```

JSON 结构（每块一条）：

```
{
  "id": "RC-ALGO-02",
  "topic": "算法",
  "source": "《开发技术文档 V1.0》§4.5；项目实践",
  "tags": ["残差", "切分", "尖峰", "退化序列"],
  "text": "**标题**\n\n 正文……"
}
```

0.4 分块原则（为什么这么切）

原则	说明
一块一个可独立回答的问题	例如"指标口径是什么"是独立一块；不要把"接口清单"和"错误码"混在一起
保留约束标记（🔒/📌/⚠）	这些标记承载"能不能改"的信息，删掉后答案会失去力度
保留编号（`BE-*` / `FR-*` / `F-*` / `T-*`）	编号是溯源锚点，也是防幻觉的抓手
保留实测值（⏱）	语料里写"性能良好"毫无价值；写"11.85 s"才能被追问
宁多勿碎	目标 150–600 字/块。过碎会让回答失去上下文，过大会稀释检索精度

1. 语料库正文

RC-OVW-01 项目是什么、要解决什么问题

- **主题**：项目概览
- **出处**：《项目任务书 V1.0.0》§1、§2
- **标签**：项目背景 目标 ERP 补货 Web 应用

项目定位：电商与仓储场景下的**销量预测 Web 应用。**运营人员在浏览器上传历史销量 CSV，系统自动完成预处理与特征工程，用 LSTM 预测未来若干天的销量，输出预测曲线与误差指标，辅助补货计划与分仓规划。

要解决的业务问题：预测偏高造成库存积压与资金占用，预测偏低导致缺货与履约延误。大促节奏叠加多仓库、多 SKU 的形态，使人工经验预估难以支撑精细化补货。

总体目标：开发一款基于 LSTM 的电商销量预测 Web 应用，用户无需掌握编程技能即可使用。

RC-OVW-02 六大功能模块

- **主题**：项目概览
- **出处**：《项目任务书 V1.0.0》§2.2、《PRD V1.0》§3
- **标签**：数据上传 特征工程 模型预测 结果可视化 历史记录 用户管理

模块	内容
数据上传	上传历史销量 CSV（商品/仓库/日期/销量），内置示例数据一键加载，校验字段完整性与时间连续性
特征工程与预处理	缺失值填补、异常值处理、归一化、滑动窗口切样；提取

模块	内容
	星期/月份/节假日/滞后/滚动均值特征
模型预测	基于 LSTM 的多步预测；以 MAE / RMSE / MAPE 评估；目标 MAPE ≤ 20%，MAE 与 RMSE 较移动平均基线降低 ≥ 15%
结果可视化	折线图展示历史 vs 预测；展示误差指标与分商品预测表格；支持导出
历史记录	保存预测历史，查看过往任务与曲线对比，支持导出
用户管理	注册、登录、个人信息管理，保障数据安全

RC-OVW-03 硬指标达成情况与实测数据

- **主题**：项目概览
- **出处**：《测试报告 V1.0》§2、《开发技术文档 V1.0》§9.7、附录 D；⏱ 本机实测
- **标签**：MAPE MAE RMSE 基线 实测 达标

指标	实测值	任务书要求	结论
加权 MAPE	**14.26%**	≤ 20%	✅ 达标
MAE 较移动平均基线	降低 **26.42%**	≥ 15%	✅ 达标
RMSE 较移动平均基线	降低 **27.52%**	≥ 15%	✅ 达标
达标序列占比	**83.95%**	≥ 60%	✅ 达标

性能实测（⏱ 本机：8 逻辑核 / 23.8 GB 内存 / MX450 2048 MiB）：

端到端 113 序列 h=14 为 **11.85 s**（SLA 60 s，余量约 5 倍）；单次批量推理 14–21 ms；

训练 45 s（CUDA）/ 80 s（CPU）；训练峰值显存 **111 MB**。

RC-ARC-01 系统架构与端口

- **主题**：架构与部署
- **出处**：《部署运维手册 V1.0》§1.1、《开发技术文档 V1.0》§2.7
- **标签**：前后端分离 B/S Nginx uvicorn 端口

前后端分离的 B/S 架构，**单机部署**：

- 生产态：Nginx（:80）托管前端静态产物 `web/dist`，并把 `/api`、`/health`、`/docs`、`/redoc`、`/openapi.json`、`/static/` 反代到后端 `127.0.0.1:8000`。
- 开发态：Vite dev server（**:5173**，`strictPort: true`，端口被占直接失败而非静默换端口）把 `/api`、`/health` 代理到后端。
- 后端：FastAPI + uvicorn，监听 **:8000**（`APP_PORT` 可覆盖）。
- 数据库：SQLite 文件型 `data/app.db`，无端口。

RC-ARC-02 后端分层结构

- **主题**：架构与部署
- **出处**：《开发技术文档 V1.0》§2.2
- **标签**：分层 api services dao models schemas algo

```
app/
├── api/v1/     路由层（auth / datasets / predictions / history / admin）
├── services/   业务服务层（8 个：auth / dataset / predict / result / history / admin / cost_model / maintenance）
├── dao/        数据访问层（8 个，按表划分）
├── models/     ORM 实体（9 个，与 9 张表一一对应）
├── schemas/    Pydantic 请求/响应模型
├── core/       配置 / DB / 错误 / 日志 / 中间件 / 模型单例 / 安全
├── algo/       算法层（9 模块，可独立于 Web 层运行）
└── static/vendor/   Swagger / ReDoc 离线资源
```

关键约定：🔒 `services/` 层**零命中训练入口**（由 AST 扫描守卫）；

🔒 `result_service` 顶层**不 import algo**（函数内为列名常量 import 是允许的）。

RC-ARC-03 技术栈与版本（版本锁定）

- **主题**：架构与部署
- **出处**：《项目任务书 V1.0.0》§3.2、《开发技术文档 V1.0》§1.2、`requirements.txt`
- **标签**：技术栈 版本锁定 `PyTorch` `React`

层	选型	版本
语言	Python	3.12+
后端框架	FastAPI / uvicorn	0.141.1 / 0.52.4
ORM	SQLAlchemy	2.0.36
校验	Pydantic	2.9.2
认证	bcrypt / PyJWT	5.0.0 / 2.9.0
深度学习	PyTorch（CUDA）	**2.9.1+cu128**
数据	pandas / numpy	2.3.3 / 2.5.3
导出	openpyxl	3.1.5
前端	React / Vite	18.3.1 / 5.4.9
UI	Ant Design / TailwindCSS	5.21.6 / 3.4.14
图表	ECharts（懒加载）	5.5.1
数据库	SQLite（WAL + 外键）	内置

🔒 `NFR-COMP-07`：**版本锁定**，编码期不得更换框架。任务书任务分解表第 6 项写「Flask/FastAPI」，但 §3.2 技术选型定的是 FastAPI → **统一按 FastAPI 执行。**

RC-ARC-04 启动自检链与运行期硬约束

- **主题**：架构与部署

- **出处**：《部署运维手册 V1.0》§1.4、§6.3、《开发技术文档 V1.0》§4.1.1
- **标签**：启动自检 `BE-05` `BE-13` `BE-15` `BE-06`

启动顺序（任一失败即**启动失败**，不做降级重试）：

1. `ensure_secret_configured()` —— 🔒 `BE-05`：`JWT_SECRET` 缺失或 < 32 字符 → 直接失败。

若不拦，服务能启动、能登录，但任何人拿公开默认密钥即可伪造 Token。

2. `init_db(run_seed=True)` —— 建表/视图/触发器 + 设置 PRAGMA + 补种子（幂等）。

🔒 必须执行 `code/db/schema.sql`，**不能用 `Base.metadata.create_all()`**

—— 后者建不出视图、触发器与游客哨兵行。

3. `load_active_model()` —— 🔒 `BE-15` 预加载模型；🔒 `BE-06` 校验 `.env` 的 `ACTIVE_MODEL`

与 `model_registry.is_active=1` 是否一致。失败即启动失败。

其他硬约束：🔒 `BE-13` 每次连接必须 `PRAGMA foreign_keys = ON`（SQLite 默认关闭）。

RC-ARC-05 接口文档离线化（Swagger / ReDoc）

- **主题**：架构与部署
- **出处**：《开发技术文档 V1.0》§10.4、《部署运维手册 V1.0》§3.3
- **标签**：`Swagger` `ReDoc` 离线化 `CDN` 白屏

`/docs`、`/redoc` 默认从 `cdn.jsdelivr.net` 加载 UI 资源。**国内受限网络下 CDN 被重置**，

症状是「HTML 拿到 200，但页面白屏、一个接口条目都渲染不出来」。

做法：随仓库自带 `swagger-ui-bundle.js`（v5.33.1）、`swagger-ui.css`、`favicon-32x32.png`、

`redoc.standalone.js`（v2.1.5），放在 `app/static/vendor/`，用自定义 `/docs`、`/redoc` 路由显式指向本地。

⚠ **两处藏在默认参数里的外链**（只改主参数除不干净，EP-10 实测命中）：

① `get_redoc_html(..., redoc_favicon_url=...)` 默认指向 `https://fastapi.tiangolo.com/img/favicon.png`；

② `with_google_fonts=True` 会外链 Google Fonts。两者都必须显式关闭/改写。

验收必须用真实浏览器：`curl` 返 200 在缺陷存在时**同样是 200。**

判据是三条一起看：`.opblock` 数量 > 0、DOM 内零外部 `http(s)` 引用、`Network.loadingFailed` 为空。

RC-ALGO-01 算法流程四阶段

- **主题**：算法
- **出处**：《项目任务书 V1.0.0》§3.3
- **标签**：流程 数据准备 训练 调参 部署

1. **数据准备**：使用菜鸟「需求预测与分仓规划」数据集（阿里天池 167097）。按时间先后 **8:1:1** 划分训练/验证/测试集（避免未来信息泄漏）；缺失值线性插值；异常销量尖峰截断；滑动窗口切分扩充样本。
2. **模型训练**：多层 LSTM，损失 MSE，Adam 优化器 + 学习率衰减，验证集早停防过拟合。
3. **模型优化**：对窗口长度、隐藏层单元数、层数、dropout、学习率做网格搜索；以**移动平均基线**对比确认提升幅度。
4. **推理部署**：模型部署到服务端，通过 API 提供预测服务。

RC-ALGO-02 五条必须保持的算法设计铁律

- **主题**：算法
- **出处**：项目实践（`app/algo/features.py`、`train.py`）；《测试报告 V1.0》§3.2

- **标签**：铁律 残差 序列切分 退化序列 尖峰 主指标口径

1. **学残差，不学绝对值**：目标 = `(y − 移动平均基线)/σ`（`TARGET_RESIDUAL`）。

学绝对值会让预测塌回序列均值（实测 预测 std/真值 std = 0.26）。

2. **按序列切分**（`split_by_series`），不按样本切 —— 同序列相邻两天近乎相同，按样本切**必然泄漏**。

3. **退化序列过滤**：`MIN_USABLE_SIGMA = 0.5`。σ≈0.03 的近常量序列除完后残差放大十倍，会污染全局 loss。

4. **营销节点保护**：`truncate_spikes(..., preserve_calendar=...)`。

无条件截断会把双十一尖峰削掉，而那是**唯一可学的业务信号**。

5. **主指标只算 L1**：`levels_for_main=("L1",)`。L2 是规则的锅（实测 L2 MAPE ≈ 60%），

混进来会把 13.6% 变成 25.9%。

RC-ALGO-03 序列三级降级（L1 / L2 / L3）

- **主题**：算法
- **出处**：《PRD V1.0》`FR-MOD-05` / `FR-MOD-09`、《开发技术文档 V1.0》§4.5
- **标签**：分级 降级 稀疏 长尾 `L1` `L2` `L3`

级别	判定规则（`L1_MIN_VALID_POINTS = 90`）	处理方式
L1	仓库 × 商品序列**有效历史 ≥ 90 天**	走 LSTM 推理
L2	自身不足但**同商品跨仓聚合 ≥ 90 天**	商品级降级 + 按各仓历史占比**分摊**
L3	跨仓聚合仍 < 90 天	🔒 **不输出数值预测**，只给历史均值

为什么 L2 用"跨仓聚合"：真实数据是长尾 —— 天池菜鸟数据集 963 商品 × 5 仓 ≈ 4,815 条理论序列，但存在 **17,063 条仓库缺失样本**，很多商品只在 1–2 个仓有货。跨仓聚合能显著提高可用序列数，且分摊比例有真实依据（各仓历史占比）。

📌 **稀疏是常态，不是异常分支** —— 三级降级是**正常路径。**

RC-ALGO-04 模型结构与超参（lstm_v1）

- **主题**：算法
- **出处**：`outputs/models/lstm_v1.report.json`；⏱ 实测
- **标签**：`LSTM` `超参` `参数量` `训练`

项	值
隐藏层单元	48
层数	2
滑动窗口	21
特征数	11
参数量	**31,729**
训练 / 验证样本	8,140 / 850
最佳轮次	第 31 轮（早停于第 51 轮）
最佳验证损失	0.4741
训练耗时	⏱ 45.26 s（CUDA）
权重文件大小	128 KB（`outputs/models/lstm_v1.pt`）

输出：多步预测（horizon 支持 7 / 14 / 30）。**基线**：移动平均。

RC-ALGO-05 为什么不做在线重训练

- **主题**：算法
- **出处**：📌 已确认决策 `Q6`；《开发技术文档 V1.0》§4.7
- **标签**：`预训练` `SLA` `重训练` `决策`

📌 **决策：纯预训练推理**，✕ 不提供在线重训练；端到端最长 **1 分钟**（硬约束）。

理由（答辩必问，务必按此回答）：**不是显存不够** ——

模型只有 31,729 个参数，⏱ 训练峰值显存实测仅 **111 MB**，2 GB 显存绰绰有余。

真正的理由是：**「训练 + 网格调参 + 早停」无法稳定塞进 60 秒预算。**

与其提供一个会经常超时的功能，不如不做，并把该限制**明确写在界面上**（参数页高级设置）。

RC-ALGO-06 预处理与特征工程细节

- **主题**：算法
- **出处**：《开发技术文档 V1.0》§4.3、§4.4
- **标签**：插值 截断 滑动窗口 特征

- **缺失值**：线性插值填补。
- **异常尖峰**：截断处理，但**保留营销日历节点**（见 `RC-ALGO-02` 第 4 条）。
- **滑动窗口**：把长序列切成 `(window=21, 目标)` 的样本对，实现样本扩充。
- **特征（11 个）**：星期、月份、节假日、滞后销量、滚动均值等。
- **按序列切分**：训练/验证/测试按**时间先后** 8:1:1，**按序列整体切分**，不按样本切。

RC-ALGO-07 评测必须"样本外"，且基线可比

- **主题**：算法
- **出处**：项目实践；《测试报告 V1.0》§2.6
- **标签**：评测 `backtest` 基线 可比性

- 评测必须 `backtest=True`（先砍掉末尾 horizon 天再评估），否则就是"看着答案出题"。

- 🔒 **基线可比性硬约束**：移动平均基线必须在**完全相同的预测点、相同序列集合**上计算。

基线算在不同的点上，提升幅度毫无意义。

- 🟤 `verify_ep5_equiv.py` 做改造前后**逐位等价性**回归：把改造前的滚动多步实现

逐字保留为 `reference_roll`，对全部 L1 序列 × 3 档步长断言 `np.array_equal`。

RC-ALGO-08 训练与调参时机（GIT-7 纪律）

- **主题**：算法
- **出处**：《开发技术文档 V1.0》§9.1、《部署运维手册 V1.0》§5.2
- **标签**：`GIT-7` 重训 同步 两套数字

🔒 **`GIT-7`**：修改算法参数后**必须**重跑：

```
python scripts/train_and_register.py
python scripts/eval_report.py --horizon 14 --export-series
python scripts/sync_eval_metrics.py      # → web/src/generated/eval_report.json
python scripts/sync_eval_metrics.py --check
```

否则界面数字会落后于模型（本项目历史上出现过"两套数字"问题）。

RC-ALGO-09 已知算法局限

- **主题**：算法
- **出处**：《项目任务书 V1.0.0》§6.3；《测试报告 V1.0》§4（遗留问题）
- **标签**：局限 新品 长尾 步长

本模型适用于**具有稳定历史规律的成熟商品**销量预测。以下情形预测能力有限：

- 无历史数据的**新品**；
- **长尾低频商品**（走 L2/L3 降级路径，L2 实测 MAPE ≈ 60%）；
- 受**突发事件**影响的销量序列；
- **误差随预测步长增加而增大**（故预测区间上限设为 30 天）。

RC-MET-01 指标口径（必须按此口径回答）

- **主题**：数据与指标口径
- **出处**：📌 已确认决策 `Q2`；《PRD V1.0》§4
- **标签**：口径 加权 `MAPE` `WAPE` 达标序列占比 基线

📌 **主指标 = 销量加权 MAPE**，即 `Σ|e| / Σ|y|`（也称 WAPE）。

辅指标 = 达标序列占比 ≥ 60%。基线 = 移动平均。

为什么用加权而非"每序列 MAPE 再平均"：后者会让一条日销 3 件的小序列与一条日销 300 件的大序列**权重相同**，指标失去业务意义。

达标序列：单序列 MAPE ≤ 20% 即视为达标。实测 83.95%（68/81）。

RC-MET-02 数据集字段与校验规则

- **主题**：数据与指标口径
- **出处**：《开发技术文档 V1.0》§4.2、《PRD V1.0》`FR-UPL-*`
- **标签**：字段 校验 七步 仓库列

必需四列（英文列名，区分大小写）：`sku_id` / `warehouse_id` / `date` / `sales`。

七步顺序校验：扩展名 → 文件大小 → 行数 → **表头完整性** → 日期可解析性 → 销量非负 → 时间连续性（断档）。

🔒 **仓库列缺失即拒**（`FR-UPL-05`）—— 因为预测粒度是「仓库 × 商品」，没有仓库维度就无法定位序列，后面所有指标都错且**看不出来**。

上限：单文件 ≤ 20 MB（`MAX_FILE_SIZE_MB`）；行数 ≤ 500,000（`MAX_ROWS`）；序列数 ≤ 1,000（`MAX_SERIES`）。

RC-MET-03 覆盖率披露口径

- **主题**：数据与指标口径
- **出处**：《PRD V1.0》`FR-VIS-04`、《开发技术文档 V1.0》§5
- **标签**：覆盖率 `L1` `L2` `L3` 披露

结果页必须披露三项序列数及其占比：`L1` / `L2` / `L3`，且 🔒 **三项之和 = `total`**。

- `L3` **不输出数值预测**，只给历史均值（`FR-MOD-09`）。
- 主指标**只统计 L1**；L2 / L3 单独披露，不混入主指标。
- 覆盖率取自 `tasks` 表的冗余列，**不是**再 `count_by_level` 重算（防口径分叉）。

RC-MET-04 已知数据事实（长尾）

- **主题**：数据与指标口径
- **出处**：联网核实；《开发技术文档 V1.0》§4.5
- **标签**：天池 `167097` 长尾 缺失

天池菜鸟「需求预测与分仓规划」数据集（ID 167097）：963 商品 × 5 仓 ≈ **4,815 条理论序列**，

约 450 天历史。**长尾**明显：大量商品仅覆盖 1–2 个仓，**缺失仓库样本 17,063 条。**

→ 结论：**稀疏是常态，三级降级是正常路径，不是异常分支。**

RC-DB-01 数据库对象总览

- **主题**：数据库
- **出处**：《数据库设计文档 V1.0》；⏱ 实测 `data/app.db`
- **标签**：`SQLite` `8 表` `视图` `触发器` `索引`

对象	数量	明细
表	**8**	`users` / `datasets` / `tasks` / `predictions` / `metrics` / `model_registry` / `operation_logs` / `events`
列	**121**	——
视图	**1**	`v_task_summary`（含 `dataset_id`）
触发器	**3**	`trg_users_updated` / `trg_datasets_updated` / `trg_tasks_updated`
索引	**12**	10 个显式命名 + 2 个约束自动创建

`PRAGMA user_version = 1`（见 `RC-DB-03` 迁移）。

RC-DB-02 PRAGMA 四件套（🔒 每次连接执行）

- **主题**：数据库
- **出处**：《开发技术文档 V1.0》§3.9、《部署运维手册 V1.0》§4.3

- **标签**：PRAGMA 外键 WAL BE-13

```
PRAGMA foreign_keys = ON;       -- 🔒 SQLite 默认 OFF，不开启则外键完全失效（BE-13）
PRAGMA journal_mode = WAL;      -- 一写多读并发（NFR-REL-04）
PRAGMA busy_timeout = 5000;     -- 写锁等待 5 秒而非立即报错
PRAGMA synchronous = NORMAL;    -- WAL 下的推荐值，兼顾安全与性能
```

⚠ journal_mode=WAL 是**库级、写一次持久生效**；而 foreign_keys / busy_timeout 是**连接级**的，每条新连接都要重设。

RC-DB-03 迁移策略（不引入 Alembic）

- **主题**：数据库
- **出处**：📌 TD-DB-03；《数据库设计文档 V1.0》
- **标签**：迁移 user_version 幂等 备份

📌 本项目**不引入 Alembic 等迁移框架**，用 **PRAGMA user_version** 手工管理。

迁移脚本必须满足：① 幂等（重复执行不报错）；② 迁移前自动备份 data/app.db；

③ 执行后设 PRAGMA user_version = 1；④ 输出前后列数对比（120 → 121）。

脚本：scripts/migrate_0001_ch01_role.py（CH-01：加 role 列 + action 扩 8 类 + 补管理员种子）。

RC-DB-04 备份与恢复（🔒 必须用 .backup）

- **主题**：数据库
- **出处**：《部署运维手册 V1.0》§7
- **标签**：备份 恢复 WAL integrity_check

🔒 **不可直接 `cp` 数据库文件**：WAL 模式下直接复制可能拿到缺 WAL 内容的不一致快照。

```
sqlite3 data/app.db ".backup 'backup/app-$(date +%F-%H%M).db'"
sqlite3 backup/app-xxx.db "PRAGMA integrity_check;"     # 期望 ok
```

恢复时必须同步删除 `app.db-wal` / `app.db-shm`，否则与恢复的库不匹配。

排障三件套：`PRAGMA integrity_check`（页面损坏）/ `PRAGMA foreign_key_check`（孤儿行）/ `PRAGMA user_version`（结构版本）。

完整备份集：`data/app.db` + `outputs/models/` + `outputs/eval_report.json` + `.env`（后者含 `JWT_SECRET`，丢失后所有已发 Token 失效）。

RC-API-01 接口总览（30 个路径）

- **主题**：接口与契约
- **出处**：`outputs/电商销量预测系统_openapi.json`；⏱ 由 `app.openapi()` 导出
- **标签**：`接口` `OpenAPI` `契约` `30`

OpenAPI 3.1，共 **30 个路径 / 32 个操作 / 78 个 Schema**。方法分布：GET 19、POST 8、PATCH 3、DELETE 2。

分组	路径前缀	接口数
系统	`/health`	1
认证	`/api/v1/auth/*`	3
用户	`/api/v1/users/me`	2
数据集	`/api/v1/datasets/*`	7

分组	路径前缀	接口数
预测	`/api/v1/predictions/*`	5
历史	`/api/v1/history/*`	5
管理端	`/api/v1/admin/*`	9

RC-API-02 统一响应包与错误码

- **主题**：接口与契约
- **出处**：《开发技术文档 V1.0》§5.1.1、§5.2
- **标签**：响应包 错误码 `code` `message`

统一响应包：`{ code, message, data }`。`code = 0` 表示成功。

关键错误码：

码	含义	码	含义
`40001`	缺少必需字段（缺列）	`40007`	文件过大
`40002`	日期无法解析	`40008`	行数超限
`40003`	销量非负（负值/非数值）	`40009`	扩展名非 .csv
`40004`	序列数超限	`40010`	空文件
`40005`	日期断档	`40011`	密码强度不足
`40006`	数据不足	`40012`	参数非法
`40101`	未登录	`40301`	无权限
`40401`	任务不存在	`40402`	数据集不存在
`40404`	模型未就绪	`42901`	触发限流

🔒 **两种拒绝不混用**：未登录 → `401`（引导去登录）；已登录但非管理员 → `40301`（引导去要权限）。

把"没登录"报成"没权限"会让用户白折腾。

RC-API-03 三条接口设计红线

- **主题**：接口与契约
- **出处**：《开发技术文档 V1.0》§5、《项目开发实施方案》§9.1
- **标签**：路由顺序 幂等 软失败

1. 🔒 **路由注册顺序**（BE-39）：字面量路径必须排在参数化路径**之前。**

`POST /history/compare` 与 `GET /history/dashboard` 必须排在 `GET|DELETE /history/{task_id}` 之前，

否则 `compare` 会被当 `task_id` 吞掉（报 `40012`）。`datasets` 同理。

2. 🔒 **软失败不返 5xx**：超时返回 `E_TIMEOUT`、序列超限返回 `E_TOO_MANY_SERIES`，

HTTP 仍是 200 + `code != 0`，并给 `completed_stage` 与三条建议。

极限规模实测：1000 序列 × h=30 → `E_TIMEOUT`，**无半成品落盘。**

3. 🔒 **幂等**：`API-07`（示例数据加载）重复调用不产生重复数据集；

`API-20`（软删除）二次调用返回相同结果，且 `predictions` + `metrics` **行数完全不变。**

RC-API-04 契约一致性守卫

- **主题**：接口与契约
- **出处**：`scripts/verify_ep9_contract.py`
- **标签**：契约 双向 `MODEL_TO_TS` 漂移

`verify_ep9_contract.py` 做**四组双向比对**：

组	比对对象
A	后端模型字段（`code/backend/schemas.py`）↔ 前端类型（`web/src/types/index.ts`）

组	比对对象
B	路由与调用点：OpenAPI `paths` ↔ `web/src/api/client.ts`
C	枚举取值域：pydantic `Literal` ↔ TS 字面量联合类型
D	常量同源：后端模块常量 ↔ TS `LIMITS` / `THRESHOLDS`

🔒 **核心机制**：`MODEL_TO_TS` 是一张**必须逐条登记的映射表**。两端任何一侧新增模型或接口

而不登记，脚本即失败 → 「忘了同步」在结构上**不可能静默通过**。

RC-SEC-01 认证与会话

- **主题**：安全与权限
- **出处**：《开发技术文档 V1.0》§6
- **标签**：`JWT` `bcrypt` `限流` `token_version`

- 密码用 **bcrypt** 哈希（含 72 字节边界处理），数据库中**无明文密码**。
- **JWT** 有效期 720 分钟；`JWT_SECRET` 必须 ≥ 32 字符（`BE-05`）。
- 🔒 **`token_version`**：改密、重置密码、停用账号都会使 `token_version + 1`，

旧 Token 立即失效。

- **登录限流**：连续失败 3 次锁定 60 秒（`LOGIN_MAX_FAIL` / `LOGIN_LOCK_SECONDS`）。

⚠ 该实现是**进程内内存**，故 `--workers` 必须为 1（多进程会使限流失效）。

- **防枚举**：登录失败不区分"用户不存在"与"密码错误"。

RC-SEC-02 数据隔离红线

- **主题**：安全与权限
- **出处**：《PRD V1.0》`NFR-SEC-02`、《开发技术文档 V1.0》§6.4

- **标签**：隔离 `user_id` `BE-ADM-01` 越权

🔒 所有业务查询**强制带 `user_id` 过滤** → 越权访问返回 `40401`（不是 `403`，

避免暴露"这个 ID 存在但不属于你"）。

📌 **唯一显式例外**：管理端 `app/api/v1/admin.py`（`BE-ADM-01`）。约束：

1. 例外只允许存在于 `admin.py` **这一个文件** —— 其余文件出现不带 `user_id` 的跨用户查询

即视为红线破坏（`verify_ep8.py` 做 **AST 扫描**而非 grep）；

2. 9 条管理端路由**全部**声明 `CurrentAdmin` 依赖，禁止"先查后判"；
3. 🔒 依赖注入必须是**注解形式** `admin: CurrentAdmin`（写成 `admin=CurrentAdmin` 会把类本身当依赖传进去）；
4. 4 类写操作全部写 `operation_logs`（`action='admin'`），且**与业务同事务**。

RC-SEC-03 游客态与哨兵账号

- **主题**：安全与权限
- **出处**：《开发技术文档 V1.0》§3.3、§5.5（`TQ-4` → `JT-10`）
- **标签**：游客 `user_id=0` 哨兵 外键

`FR-USR-08` 要求未登录也能跑到结果页，但 `datasets.user_id` 是 `NOT NULL`。

📌 解法：**`user_id = 0` 表示游客**，并在 `users` 表落一条 `id = 0` 的**哨兵账号**。

哨兵的存在**不是为了"让游客能登录"**，而是为了在 `PRAGMA foreign_keys = ON` 下

让 `user_id = 0` 能通过外键校验。

游客限制：可上传、可建任务、可看结果；**导出 CSV 返回 `401`**（导出含完整明细，不应匿名可取）。

游客数据 TTL 24 小时优先清理。

RC-SEC-04 安全加固清单

- **主题**：安全与权限
- **出处**：《部署运维手册 V1.0》§4.4、《项目任务书 V1.0.0》§6.2
- **标签**：`HTTPS` `加固` `审计` `权限`

项	要求
HTTPS	🔒 **必须**（任务书 §6.2 数据隐私要求）；Nginx 已透传 `X-Forwarded-Proto`
`JWT_SECRET`	48 字符随机串，**仅存服务端**；泄露 = 可伪造任意用户 Token
`APP_ENV`	生产设 `production`，不向前端暴露内部异常栈
`CORS_ORIGINS`	生产同源反代下可留空/填真实域名
文件权限	`data/`、`.env` 仅服务账号可读写（`chmod 600 .env`）
上传限制	Nginx `client_max_body_size` > `MAX_FILE_SIZE_MB`，两处不一致会出现"前端能选、Nginx 先 413"
审计	管理端 6 类子动作全部留痕：`enable` / `disable` / `role_change` / `reset_pwd` / `hard_delete` / `switch_model`，`detail` 格式 `sub=..;target=..;by=..`

RC-PERF-01 性能基线与 SLA

- **主题**：性能与容量
- **出处**：⏱ `outputs/ep9_perf_report.json`；《开发技术文档 V1.0》§9.7
- **标签**：`SLA` `端到端` `首屏` `并发`

指标	实测	SLA / 期望

指标	实测	SLA / 期望
端到端（113 序列，h=14）	**11.85 s**	≤ 60 s（`TASK_BUDGET_MS`）
其中推理阶段	14–21 ms	——
CPU 回退（无 GPU）	23.49 s	≤ 60 s
数据集预览接口	55.7 ms	≤ 1 s
历史列表首屏	61.4 ms	≤ 1 s
首屏 FCP（生产构建）	504 ms（Edge）/ 140 ms（Chrome）	≤ 3 s
3 任务并发（各自）	32.2 / 37.6 / 37.5 s	全部成功、锁冲突 0
极限规模（1000 序列 × h=30）	250.3 s → `E_TIMEOUT`	**软失败**：无 5xx、无半成品

RC-PERF-02 容量估算

- **主题**：性能与容量
- **出处**：《部署运维手册 V1.0》§10.2、《数据库设计文档 V1.0》
- **标签**：容量 增长 百万行

维度	规模
预测明细	约 60–120 字节/行（113 序列 × 14 天 ≈ 1,582 行 ≈ 180 KB）
数据集	约 40 字节/行（500k 行上限 ≈ 20 MB，与 `MAX_FILE_SIZE_MB=20` 对齐）
主要增长源	`predictions` 表
重评阈值	🔒 `TD-DB-01`：`predictions` 超过 **100 万行**需重新评估索引与分页

RC-PERF-03 数据清理与 TTL 三档

- **主题**：性能与容量

- **出处**：《数据库设计文档 V1.0》（JT-10 / JT-11 / DB-12）
- **标签**：TTL 清理 exports 游客

目录 / 对象	TTL	说明
data/exports/（正式导出）	**7 天**（EXPORT_TTL_DAYS）	用户可下载的导出产物
data/exports/tmp/（错误明细临时文件）	**1 小时**（EXPORT_TMP_TTL_SECONDS）	⚠ **必须与正式导出分目录**，否则 7 天策略会误保留临时文件
游客数据（user_id = 0）	**24 小时**（GUEST_DATA_TTL_HOURS）	优先清理

运维入口：python scripts/cleanup_tmp.py --with-tmp --with-guest（先加 --dry-run 预览）。

RC-PIT-01 最易踩的坑之一：副作用被异常回滚

- **主题**：踩坑与排错
- **出处**：《项目开发实施方案 V1.0》附录（十八条坑 第 4 条）
- **标签**：get_db rollback commit 隐式

🟤 **现象**：services/ 里做完状态变更后 raise BizError，指望 get_db 提交 ——

但 get_db 的契约是"**异常 → rollback**"，变更**被回滚。**

三次真实后果：

- EP-2：登录限流**永不生效**（fail_count 每次归零）；
- EP-4：五类校验失败**一条记录都不落库**，API-11 的错误明细永远拿不到；
- EP-5：**超时软失败任务查不到。**

防法：需要保留的副作用**在抛错前显式 db.commit()。**

判据：这条缺陷**在单次请求的返回值上看不出来** —— 接口该报的错都报对了，

只有再查一次库才露馅（原话是"复现得了但查不到"）。

RC-PIT-02 最易踩的坑之二：哨兵值

- **主题**：踩坑与排错
- **出处**：《项目开发实施方案 V1.0》附录（十八条坑 第 5 条）
- **标签**：哨兵值 限流 `datetime.min` `None`

🟤 用 `datetime.min` / `0` 之类的"哨兵时间"表示"未锁定" →

判断"是否已解锁"的 `remain <= 0` 会**同时命中"根本没锁"** → 误删正在累积的计数 →

限流静默失效（`EP-2` 实测）。

防法：「计数中」与「已锁定」必须用**不同的类型**表达（如 `None` vs `datetime`）。

RC-PIT-03 最易踩的坑之三：正则扫源码不先剥注释

- **主题**：踩坑与排错
- **出处**：《项目开发实施方案 V1.0》附录（十八条坑 第 8 条）
- **标签**：守卫 注释 `AST` 噪音

🟤 拿正则扫源码做"痕迹/字面量"守卫，却**不先剥注释** → 注释里写「已移除 `X`」反而被判违规

→ 守卫变成噪音 → 最后被人关掉。

本项目连踩三次（`EP-1`、`EP-3`、`EP-5`）。

防法：守卫一律**先剥 `//` 与 `/* */`** 再扫；**能用 AST 就别用正则**

（`EP-4` 已改用 `ast.walk` 取真实 import）。

RC-PIT-04 环境类坑（换机器会重现不了，必须文档化）

- **主题**：踩坑与排错
- **出处**：《测试报告 V1.0》§3、《部署运维手册 V1.0》§11
- **标签**：代理 `profile` 锁 隐藏属性 异步删除

坑	症状	防法
系统设了 `HTTP_PROXY`	浏览器把 `127.0.0.1` 也塞进代理 → 本地页面全白/502	浏览器加 `--no-proxy-server` + `--proxy-bypass-list=<-loopback>`
vite 默认只监听 `[::1]`	代理连不上 → 502	显式 `--host 127.0.0.1`
vite 代理写死端口	用独立临时库跑测试时，前端能开但**每个接口都失败**，后端日志无请求	改读 `VITE_API_TARGET` 环境变量
复用固定 `--user-data-dir` + 只 kill 父进程	子进程持有 profile 锁 → 新实例**静默挂起**：不报错、不产截图、空转 15 分钟	① 每次全新 profile；② 杀**进程树**；③ 单次截图套超时
`taskkill` 异步，紧接着删 profile	文件锁未释放 → 删除静默失败 → **永久残留 1.65 GB**	删除前等待 + 重试 + 失败显式告警
平台把 `rmSync` 转成异步移入回收站	"删完立刻查"**误判成清理失败**（1 分钟后为 0）	验收清理必须**隔一会儿再统计**
Chromium profile 是**隐藏属性**	`Get-ChildItem` 默认不返回 → 计数偏小 → 误判"删干净了"	统计加 `-Force`

⚠ **教训**：「删除失败」与「删除成功但看不出来」是**两种截然不同的故障**，而它们的表现都是「文件还在」。

2. 用于构建 RAG 助手的提示词

2.1 系统提示词（RAG 助手人设与边界）

```
你是「电商销量预测系统」的项目答疑助手。你的唯一知识来源是检索到的语料块。

【回答规则】
1. 只依据检索结果回答。若检索结果不足以回答，直接说「现有资料未涵盖此问题」，
   并指出最接近的语料块编号。**不得凭常识推测或编造。**
2. 每个关键结论后必须标注来源编号，格式：`（见 RC-XXX-NN）`。
3. 遇到带 🔒 的内容，必须原样保留该标记并说明「这是硬约束，改动会破坏验收」。
   遇到 📌 说明「这是已确认决策，不再讨论」。
4. 遇到具体数值（MAPE / 耗时 / 参数），必须**原样引用实测值**，
   禁止改写成「较好」「很快」这类模糊表述。
5. 若用户问的是「能不能改 X」：
   - 先查是否有 🔒 / 📌 标记；
   - 有则回答「不建议/不可以改动，原因是……」并给出依据编号；
   - 无则给出改动影响面（会牵连哪些模块、哪些校验脚本会失败）。
6. 若用户的问题涉及**其他项目或通用知识**（与本项目无关），礼貌说明本助手只回答本项目相关问题。
7. 不确定的事，明确说「不确定」。**宁可少答，不可错答。**

【语气】简洁、技术化、直接给结论。不要客套开场白，不要说「很好的问题」。
【语言】简体中文。
```

2.2 检索提示词（查询改写 / 扩展）

RAG 的第一道关口是**召回**。用户的自然语言提问往往与语料措辞不重合，

故在检索前做一次改写，用 LLM 产出多个检索式：

```
你是检索查询改写器。用户会用日常语言提问，但知识库使用技术术语。
请把用户问题改写成 3–5 条**不同角度**的检索查询，以提高召回率。

要求：
1. 覆盖：① 术语化改写（用项目里的正式说法）② 同义词/别名 ③ 若涉及编号，
   直接给出可能的编号前缀（如 BE-、FR-、RC-）；
```

```
2. 每条查询独立、简短（≤ 20 字），不要句子；
3. 只输出 JSON 数组，不要解释。

示例：
用户问题：「预测准不准？」
输出：["加权 MAPE 实测值","MAE RMSE 相对移动平均基线 降低幅度",
       "达标序列占比","技术指标 验收标准","RC-MET-01"]

用户问题：
"""
{question}
"""
```

推荐检索配置（本项目语料规模下足够）：

项	建议
召回方式	**混合检索**：BM25（`tags` + 标题）与向量检索各取 top-8，合并去重
元数据过滤	先按 `topic` 过滤（若问题明确指向某主题，如"性能"）→ 再检索
重排	用 `tags` 命中数 + 向量相似度加权（简单有效）
上下文	取 top-4 块，按 `id` 升序拼接（保持主题连贯）

2.3 回答生成提示词（带引用）

```
请基于以下检索结果回答用户问题。

【检索结果】
{context}

【用户问题】
{question}

【输出格式】
- 先用一句话给结论；
- 再用 2–5 条要点展开，每条要点末尾标注来源 ` （见 RC-XXX-NN）`；
```

```
- 若涉及代码/命令/配置，用代码块原样给出；
- 若检索结果中有 🔒 或 📌 标记，必须保留标记并说明其含义；
- 若检索结果不足以回答，输出：「现有资料未涵盖此问题。最接近的是 RC-XXX-NN，
  它讲的是……」，不要编造。
```

2.4 评测提示词（RAG 质量自检）

RAG 系统本身也需要被评测，否则无法判断"答得对不对"。以下三组用于自动化评测：

① 忠实度（有没有编）

```
判断「回答」中的每一条事实性陈述是否都能在「检索结果」中找到依据。

【检索结果】{context}
【回答】{answer}

输出 JSON：
{"total": <陈述总数>, "supported": <有依据条数>,
 "unsupported": [{"claim": "……", "why": "检索结果中无此内容"}]}

判定标准：
- 数值必须逐位一致（14.26% ≠ 约 14%）；
- 编码必须一致（BE-13 ≠ BE-15）；
- 「推测/推断」的表述也算无依据。
```

② 检索相关性（召回得对不对）

```
给定用户问题和检索到的 4 个语料块，判断每个块与问题的相关性。

【问题】{question}
【语料块】{chunks}    （每条含 id / topic / tags / 正文）

对每个块输出 0–2 分：
  2 = 直接回答该问题
```

```
  1 = 提供背景，但不直接回答
  0 = 无关

输出 JSON 数组：{"id": "RC-XXX-NN", "score": 2, "why": "……"}
```

③ 答案完整性（该说的说了没）

```
给定用户问题、标准答案要点、以及 RAG 的回答，判断要点覆盖率。

【问题】{question}
【应覆盖要点】{gold_points}
【RAG 回答】{answer}

输出 JSON：
{"covered": ["要点 1", "要点 2"], "missed": ["要点 3"],
 "coverage_rate": 0.67, "note": "遗漏的要点是否影响可用性"}
```

2.5 一组"必须答对"的验收问题（golden set）

用于回归 RAG 质量。以下每个问题都应能由语料库正确回答：

#	问题	期望命中的语料块	关键判据
1	为什么不做在线重训练?	`RC-ALGO-05`	必须说明**理由是 SLA 而非显存**，并给出 111 MB 实测
2	预测粒度是什么？为什么仓库列缺失要拒?	`RC-MET-02`	必须答「仓库 × 商品」，且说明"错了看不出来"
3	指标口径是加权 MAPE 还是平均 MAPE?	`RC-MET-01`	必须答 WAPE 及其理由
4	L2 序列的判定规则是什么?	`RC-ALGO-03`	必须提到**跨仓聚合 ≥ 90 天**
5	关键坑：为什么接口报错了	`RC-PIT-01`	必须提到 `get_db` 异常回滚

#	问题	期望命中的语料块	关键判据
	但库里没记录?		+ 显式 commit
6	前端能开但所有接口都失败，后端日志没请求，为什么?	`RC-PIT-04`	必须提到 vite 代理端口写死 / `VITE_API_TARGET`
7	为什么数据隔离红线只能在 `admin.py` 例外?	`RC-SEC-02`	必须提到 AST 扫描 + `CurrentAdmin` + 全审计
8	`/docs` 白屏但返回 200，怎么排查?	`RC-ARC-05`	必须提到 CDN 被墙 + 三条判据

2.6 常见的 RAG 失败模式（本项目语料下要特别注意）

失败模式	症状	缓解
数值被"约化"	把 14.26% 答成"约 14%"	系统提示词第 4 条 + 忠实度评测
约束标记被丢掉	把 🔒 硬约束答成"建议"	系统提示词第 3 条；语料块保留标记
跨块混淆	把 `BE-05`（密钥）与 `BE-13`（外键）搞混	检索后按 `id` 升序拼接 + 忠实度评测查编号
答了"通用最佳实践"	不问项目，直接讲"一般推荐用 Redis"	系统提示词第 6 条（只答本项目）
过度自信	检索为空仍编出答案	系统提示词第 1 条 + 第 7 条（宁可少答）

---

3. 语料库的可复现性与自检

```
python scripts/build_rag_corpus.py           # 抽取 → outputs/电商销量预测系统_rag_corpus.json/.jsonl
python scripts/build_rag_corpus.py --check   # 自检：块数 / id 唯一 / 元数据齐备 / 标签非空
```

自检项：① 块数 ≥ 40；② `id` 全局唯一且符合 `RC-<主题>-<序号>` 格式；

③ 每块 `topic` / `source` / `tags` / `text` 均非空；④ `topic` 取值在白名单内；

⑤ 正文长度在 80–2000 字之间（过短无信息量、过长稀释检索精度）。

文档结束。相关文档：《开发技术文档 V1.0》（约束编号体系源头）、

《项目开发实施方案 V1.0》（十八条坑与阶段判据）、《测试报告 V1.0》（缺陷记录，

语料块 `RC-PIT-*` 的原始出处）。